

Identity Governance Validation Engine

Governance and Compliance Architecture

Microsoft Entra ID | Azure Functions | PowerShell

Henry Ezihe

Identity and Access Management Engineer

May 2026

Table of Contents

- [1. Executive Summary](#)
- [2. What the Engine Catches](#)
- [3. Design Principles](#)
- [4. Scan Modes](#)
- [5. Rule Set](#)
- [6. Separation of Duties](#)
- [7. Architecture](#)
- [8. JML Engine Integration](#)
- [9. Real-World Findings](#)
- [10. Capability Summary](#)
- [11. Permissions Required](#)
- [12. Operational Boundaries](#)
- [13. Project Roadmap](#)

1. Executive Summary

Enterprise identity environments accumulate risk without surfacing it. Employees change roles and keep old access. Contractors end up in management-tier groups. Azure subscriptions accumulate Owner assignments that were supposed to be temporary. Accounts stay active weeks after people leave. None of this appears in routine reports because cloud platforms record state. They do not evaluate it against any standard.

Periodic audits catch some of it. A quarterly spreadsheet review tells you what the state was on the day someone ran the export. The day after, a new direct Owner assignment can appear and go undetected for three months.

This engine is a compliance and risk assessment system for Microsoft Entra ID and Azure that evaluates every identity in the tenant against a configurable rule set and produces structured, machine-readable output. It runs in three modes: as a pre-provision gate before any access is granted, as a post-provision verification after provisioning completes, and as a scheduled scan across all tenant users.

It integrates directly with the JML Identity Lifecycle Engine. The Python provisioning pipeline calls it over HTTP at two points in every provisioning event. The two systems are decoupled at the HTTP boundary and can also run independently.

27 governance rules across seven categories. Separation of Duties detection against a configurable conflict catalogue. Risk scoring with suppression to prevent severity inflation. Every run produces the same output format. Risk trends are tracked across runs.

2. What the Engine Catches

At Pre-Provision

A Contractor being provisioned into a manager-tier group. A missing manager association. A duplicate UPN. An employment type that conflicts with the requested role. All of these are evaluated against the canonical identity payload before the Graph API is called, using a synthetic snapshot with no Graph calls and no side effects.

At Post-Provision

Whether the provisioned user actually received the expected groups. Whether any group membership violates the entitlement model's employment type policy. Whether MFA is registered on privileged accounts. Separation of Duties conflicts in the real provisioned state. HYG-* hygiene findings are recorded as warnings rather than failures because a freshly created account has never been signed into and cannot have MFA registered yet.

During Full Scan

All of the above across every user in the tenant, plus RBAC findings: direct role assignments, Owner assignments at subscription scope, and cross-plane correlation between privileged group membership and subscription-level RBAC roles. SoD conflicts that entered the tenant through any means, including manual assignment or processes outside the JML pipeline, are detected here.

During Drift Only

Structural rules only: RBAC, Architecture, and Correlation. New Owner assignments and group structure changes are detected without running a full evaluation. This is the daily lightweight check between full scans.

3. Design Principles

The architecture reflects a set of explicit decisions. Each one has a trade-off that was chosen deliberately.

Principle	What It Means in Practice
Collectors are fact-only	IDCollector and RbacCollector call the API, normalize the response, and return a structured snapshot. They contain no rule logic and no interpretation of what the data means. A bug in a collector is a data bug. A bug in a rule processor is a logic bug. The two failure modes are independently testable.
Deterministic rule processing	The same input always produces the same output. Rules are evaluated from a static configuration file. There is no randomness, and no state carried between evaluations. If a finding appears in one run and disappears in the next with no change to underlying state, that is a defect.
Transparent risk scoring	Every finding carries its weight explicitly. The risk score formula is documented and uses a fixed normalization constant. An IAM engineer can look at any entity's findings, sum the weights, apply the formula, and arrive at the same score the engine produces. There is no black-box weighting.
Suppression over inflation	A direct Owner assignment at subscription scope could fire multiple rules simultaneously. Without suppression, one security fact inflates the score several times. The suppression hierarchy is declared in Rules.Json, not hardcoded. The most specific rule suppresses the less specific ones for the same entity.
Separation of compliance and hygiene	Hygiene signals are real risk indicators, but they are not compliance failures in the same category as a terminated employee with active group memberships. Conflating them inflated compliance statistics in early runs.

Principle	What It Means in Practice
	Hygiene findings contribute to the risk score but do not affect the compliance gate.
Scope-aware failure handling	The engine documents exactly what happens when each required API scope is missing. Most tools fail silently or return empty results. The explicit failure mode documentation for AuditLog.Read.All and UserAuthenticationMethod.Read.All came from production failures encountered during development.

4. Scan Modes

The engine runs in five modes. Each mode targets a different operational need and uses a different collection path.

Mode	What It Evaluates	When It Runs
FullScan	All identities, all 27 rules. Full RBAC scanning across all subscriptions. SoD evaluation against real group memberships.	Weekly compliance baseline, monthly audit reporting, drift trend analysis
PreProvision (payload)	Single identity. Blocking rules only. No Graph API calls. Synthetic snapshot from canonical HR payload. SoD skipped.	JML Joiner gate before any Entra ID object is created. Returns pass or fail in under ten seconds
PreProvision (userId)	Single real identity. Three Graph calls. Blocking rules plus SoD against actual tenant state.	JML Mover gate, ad hoc identity review
PostProvision	Mapped internally to PreProvision with targetUserId. Fetches real provisioned object. HYG-* findings demoted to warnings.	JML post-provision verification. Runs seconds after provisioning completes
DriftOnly	Architecture, RBAC, and Correlation rules only. Does not evaluate individual identity hygiene or entitlements.	Daily lightweight check between full scans. Detects new Owner assignments and structural changes

PostProvision is not a separate mode value in the engine. The HTTP trigger maps a PostProvision request to the PreProvision mode with a targetUserId. The engine fetches the real provisioned object and evaluates it. HYG-* findings are demoted to warnings in this path.

The engine scans every Azure subscription the service account can see, not a pre-configured list. An IAM risk in a subscription the IAM team does not know about is still a real risk. Scope restriction to a specific subscription is available as an opt-in parameter.

5. Rule Set

27 governance rules across seven categories. Each rule has a stable rule ID for downstream automation and SIEM correlation. Blocking rules prevent provisioning from proceeding in PreProvision mode. Non-blocking rules contribute to the risk score without affecting the compliance gate.

Category	Rules	Blocking
Identity	IDENT-001, IDENT-002, IDENT-003, JOIN-001, JOIN-002	IDENT-001, IDENT-002, JOIN-001, JOIN-002
Access	ACCESS-001, ACCESS-002, ACCESS-003, ENT-001, ENT-002, ENT-003, ENT-004, PIM-001	ACCESS-002, ENT-002, ENT-004
Architecture	ARCH-001, ARCH-002, ARCH-003	ARCH-001
Hygiene	HYG-001, HYG-002, HYG-003, HYG-004	HYG-004 (demoted to warning in PostProvision)
RBAC	RBAC-001, RBAC-002, RBAC-003	RBAC-003
Correlation	CORR-001, CORR-002, CORR-003, CORR-004, CORR-005	CORR-001, CORR-002, CORR-003
SoD	SOD-EVAL-001 (pointer to sod_policies.json conflict pairs)	Block-level conflict pairs in sod_policies.json

Dual Classification Model

Most scoring systems produce a single number. This engine produces two independent outputs per identity because compliance and risk answer different questions.

ComplianceStatus (Compliant, NonCompliant, or Blocking) is a binary gate for automated workflow decisions. A JML provisioning event needs a yes or no answer, not a score.

RiskScore (0 to 100, normalised) is a continuous signal for security triage and trend analysis. Of all identities that have issues, which need attention first? RiskDensity, calculated as RiskScore divided by violation count, distinguishes a single severe finding from a broad spread of lower-severity ones.

Keeping these separate means an identity with an inactive account and a stale password shows as Compliant with hygiene signals, not as NonCompliant. This distinction matters when compliance statistics are reported to leadership.

Suppression Chain

A direct Owner assignment at subscription scope could fire multiple rules simultaneously. Without suppression, one security fact inflates the risk score several times. The suppression hierarchy is declared in Rules.json, not hardcoded in the evaluation logic. The most specific rule suppresses the less specific ones for the same entity. Owner at subscription scope fires RBAC-003, which suppresses RBAC-002 and RBAC-001 for that assignment. The score reflects one finding.

Risk Score Formula

$\text{RiskScore} = \text{Min}(\text{RawWeightSum} / 900 \times 100, 100.0)$

Rule weights range from 15 (Low) to 100 (Critical). SoD block violations carry weight 100. SoD warn violations carry weight 70, configurable in Rules.json. The 900 cap keeps scores comparable across runs and tenants.

6. Separation of Duties

SoD conflict detection was added in Phase 2.5. It operates as two independent controls reading from the same policy file.

How the Evaluation Works

Evaluate-SoDConflict in IDRuleProcessor.ps1 loads sod_policies.json and evaluates each user's group memberships against every conflict pair. The intersection logic is ANY_TO_ANY: a violation fires if the user holds at least one group from set_a and at least one group from set_b simultaneously. Block policies produce Critical blocking findings. Warn policies produce High non-blocking findings.

Each SoD finding carries the compound rule ID SOD-EVAL-001 followed by the policy ID, so findings are traceable to both the rule that detected the conflict and the policy that defined it.

Two Controls, One Policy File

The Python sod_checker.py in the JML engine is the preventive control. It runs before any Graph API call. A block violation stops provisioning before the user object exists.

The PowerShell Evaluate-SoDConflict is the detective control. It runs during FullScan and post-provision verification, evaluating real tenant group memberships against the same catalogue. This catches violations that entered the tenant through manual assignment or before SoD was active.

sod_policies.json is the single source of truth. Policy definitions are not duplicated. Adding a new conflict pair requires one JSON edit and no code changes in either runtime.

Policy	Conflict Pair	Risk	Action
SOD-001	Payment Approver + Payment Processor	Critical	Block
SOD-002	IT User Provisioner + IT Access Approver	Critical	Block
SOD-003	Payment Processor + Finance Auditor	High	Warn
SOD-004	Payment Approver + Finance Auditor	High	Warn
SOD-005	Journal Poster + Finance Auditor	High	Warn
SOD-006	HR Salary Admin + HR Data Export	High	Warn

Fail-Closed on Mover

If the Graph API call to fetch current group memberships returns incomplete results for a Mover event, the SoD check blocks immediately without evaluating policies. A partial effective access set can miss real violations. A false block is recoverable through human review. A missed violation is not.

7. Architecture

The engine is structured in five layers. Each layer has one responsibility and a defined output contract. No layer reaches into the responsibility of another.

Layer	Components	Responsibility
Collection Layer	IDCollector.ps1, RbacCollector.ps1	Calls Microsoft Graph API and Azure ARM API. Returns a normalised identity and RBAC snapshot. No rule logic, no filtering, no interpretation.
Rule Evaluation	IDRuleProcessor.ps1, RbacRuleProcessor.ps1, CorrelationRuleProcessor.ps1	Evaluates findings against the rule set. Each processor handles a distinct rule category. FindingSchema.ps1 provides the shared finding structure and runs the suppression pass.
Classification	RiskClassifier.ps1	Takes the findings array and produces EntityRiskStates, a summary, and a pass or fail decision. Behaviour varies by mode.
Output	ReportGenerator.ps1	Writes JSON, CSV, drift state, and console output. ISO 8601 timestamps throughout. Drift state persists across runs for trend analysis.

Layer	Components	Responsibility
JML Integration	ValidateIdentity/run.ps1	HTTP trigger. Accepts PreProvision and PostProvision requests from the JML engine. Routes to the correct collection path and returns a structured response.

Collection Layer Detail

Three collection paths serve different modes. Get-IdentitySnapshot fetches all users, groups, and memberships for FullScan. Get-UserSnapshot makes three targeted Graph calls for a single identity check in post-provision and ad hoc review scenarios. New-IdentitySnapshotFromPayload builds a synthetic snapshot from the canonical HR payload with no Graph calls, used for pre-provision gate evaluation.

RbacCollector queries every Azure subscription the service account can see and returns all role assignments across all scopes. This is what enables the cross-plane correlation rules: a user's Entra group membership and their Azure RBAC assignments are evaluated together.

Privilege Classification

A group is classified as privileged if its tier is Manager or Manager/Staff in the entitlement model, or if its privileged flag is explicitly set to true in Rules.json. The two-source model catches groups like SG_Security_Core, which carries a Base tier but is operationally privileged. Both IDRuleProcessor.ps1 and CorrelationRuleProcessor.ps1 read the flag directly from the entitlement model.

8. JML Engine Integration

The validation engine runs as a separately deployed PowerShell Azure Function. The JML Python provisioning pipeline calls POST /api/validate at two points in every provisioning event: once before the Entra ID user object is created, and once after provisioning completes. The two systems are decoupled at the HTTP boundary.

Pre-Provision Gate

The JML engine sends the canonical identity payload in the request body. The validation engine builds a synthetic snapshot with no Graph calls and evaluates it against the blocking rules. The response carries a passed flag, a failures array with rule IDs and details, a warnings array, and a matchedRuleIds list. If passed is false, the JML engine routes the event to the hold queue and does not proceed.

Post-Provision Verification

The JML engine sends the Entra object ID of the provisioned user. The validation engine fetches the real user object with three targeted Graph calls and evaluates actual directory state against expected entitlements. HYG-* findings are demoted to warnings in this path. Discrepancies between expected and actual state are returned in the failures array.

Phase	JML Capability	Validation Engine Role	Status
Phase 0	Foundation	Data contracts, normalisation, architecture established	Complete
Phase 1	Joiner	PreProvision gate, PostProvision gate, ENT-004, ENT-002	Complete
Phase 2	PIM Eligibility	PIM-001 rule, PimSchedules in Get-UserSnapshot, graceful P2 fallback	Complete
Phase 2.5	SoD	Evaluate-SoDConflict, SOD-EVAL-001, sod_policies.json, FullScan + PostProvision	Complete
Phase 3	Mover	Delta recalculation gate, entitlement diff validation	Designed
Phase 4	Leaver	Post-revocation verification, full access removal confirmation	Designed

9. Real-World Findings

The following findings came from the first stable run against a live tenant with 48 identities, 65 groups, and two Azure subscriptions. They illustrate the risk categories the engine surfaces and why the findings are actionable rather than investigative.

Finding 01 Dormant Privileged Account with Owner Role

An account with no recorded sign-in held direct Owner role assignments on two Azure subscriptions, had no MFA registered, and was a member of a privileged-tier group. Risk score: 80.6. Compliance status: Blocking.

Five rules fired: HYG-004 (no MFA on a privileged account, blocking), RBAC-003 (Owner directly assigned at subscription scope on two subscriptions, blocking), CORR-001 (privileged group member with Owner at subscription scope, blocking), CORR-005 (account never signed in but holds RBAC assignments), and ACCESS-003 (account holds both privileged and standard group membership simultaneously).

An account that has never signed in holds direct Owner on two subscriptions. If the account credentials were compromised, the attacker would have unrestricted access to all resources in

both subscriptions with no MFA barrier. This is the highest-risk finding category the engine detects.

Finding 02 Contractors in Management-Tier Groups

Eight entities flagged by ENT-002. Contractors and interns provisioned into groups designated as Manager or Manager/Staff tier. Compliance status: Blocking.

The entitlement model defines which employment types are permitted in which groups. Exec_Team and Finance_Managers are Manager-tier groups. Only full-time employees are permitted. When a Contractor is a member of either group, ENT-002 fires with blocking status.

Contractors with Manager-tier access have visibility into resources and data that their employment classification does not authorise. Depending on the group, this includes financial systems, HR data, or privileged Azure resource management.

Finding 03 Sign-In Telemetry Gap Masking Real Risk

First production run: HYG-002 (inactive over 180 days) fired on 43 of 48 identities. The accounts were not inactive. LastSignInDateTime was null for all users because AuditLog.Read.All was missing from the Graph connection.

The engine was returning data that looked like a compliance problem but was a data collection gap. Without explicit documentation of the AuditLog.Read.All dependency and its failure behaviour, this would have appeared as a zero-percent compliant tenant and triggered remediation work on a false signal.

The HYG-002 rule documents the null LastSignInDateTime behaviour. The signInActivity capability probe in the collector detects whether the scope is available before user enumeration runs. This makes the failure detectable before the full scan completes.

Finding 04 RBAC Sprawl Across Two Subscriptions

22 direct Azure role assignments detected across two subscriptions. Policy requires group-based assignment. Six of the 22 assignments involved the Owner role assigned directly to users.

Direct role assignments bypass group-based access controls. A user with a direct Owner assignment is not visible to anyone reviewing group memberships. When that user leaves the organisation and their groups are revoked, the direct RBAC assignment persists. CORR-002 detects exactly this pattern.

All 22 assignments are flagged as RBAC-001 findings. The six Owner assignments are flagged as RBAC-003 (blocking). The suppression chain prevents the same Owner assignment from also firing RBAC-001 and RBAC-002, keeping the score accurate.

10. Capability Summary

Capability	Evidence in This Project
Architectural separation of concerns	Five distinct layers with explicit contracts. No layer reaches across boundaries. Collection, evaluation, classification, output, and JML integration are independently testable components.
Cloud IAM domain knowledge	Practical understanding of Entra ID user attributes, group privilege classification, Azure RBAC scope hierarchy, employment lifecycle states, MFA registration APIs, SoD conflict modelling, and Graph scope requirements.
Risk modelling	Dual-model classification: ComplianceStatus as a binary gate, RiskScore as a continuous signal. Suppression chain design to prevent severity inflation. Fixed normalisation cap for cross-tenant comparability.
Production constraint awareness	Documented failure modes for every API scope dependency. signInActivity capability probe. Non-fatal versus fatal error classification. Graceful degradation on missing scopes.
Audit-grade system design	Deterministic output, reproducible findings, transparent scoring formula, stable rule IDs for downstream automation, ISO 8601 timestamps for SIEM ingestion, drift state for trend analysis.
Diagnosing real-world problems	Identified and resolved 0% compliance distribution caused by a missing Graph scope. Identified double-fire on IDENT-003 and IDENT-004. Identified RBAC-002 and RBAC-003 scope stacking. Each diagnosis required reading structured output and tracing to root cause in data or logic.
SoD engine design	Preventive SoD check in the Python JML engine and detective SoD check in the PowerShell engine both read from the same sod_policies.json. One policy file. Two independent enforcement points. Conflict pairs added with no code changes.

11. Permissions Required

The following Graph API and Azure scopes are required. Each scope is documented with its failure behaviour so that missing permissions produce detectable, explainable output rather than silent gaps.

Scope	Purpose
User.Read.All	Fetch user objects and attributes

Scope	Purpose
Group.Read.All	Fetch group objects and membership
Directory.Read.All	Fetch role assignments and directory objects
RoleManagement.Read.All	Read Entra role definitions and assignments
AuditLog.Read.All	Retrieve LastSignInDateTime for HYG-001 and HYG-002
UserAuthenticationMethod.Read.All	Resolve MFA registration state for HYG-003 and HYG-004
PrivilegedAccess.ReadWrite.AzureADGroup	PIM group eligibility (Phase 2)
PrivilegedEligibilitySchedule.ReadWrite.AzureADGroup	PIM eligibility schedule requests (Phase 2)

12. Operational Boundaries

The boundaries below define what the current implementation covers and what it does not. Stating these explicitly is part of honest architecture documentation.

Boundary	Description
Read-only engine	The engine detects and reports. It does not remediate. Remediation handlers that consume the JSON output are the intended next layer.
FullScan scales as O(groups x members)	Get-IdentitySnapshot fetches all users, groups, and memberships sequentially. Get-UserSnapshot resolves this for single-user checks. Full tenant parallelisation is documented but not yet built.
SoD logic duplicated across runtimes	The ANY_TO_ANY intersection check exists in both PowerShell and Python. Policy definitions are not duplicated, only the evaluation logic. The long-term path is a Python timer function calling sod_checker.py directly.
PIM requires Entra ID P2	PIM schedule cmdlets and eligibility schedule requests require P2. The engine skips PIM evaluation rather than failing when the licence is absent.
PIM schedule propagation lag	PIM eligibility assignments can take 15 to 30 seconds to appear in Graph after creation. PIM-001 is non-blocking for this reason.
Fixed scoring cap	The 900 normalisation cap needs updating if new rules push the maximum possible weight above it. Cap derivation from the active rule set is a planned improvement.

Boundary	Description
Entitlement model is manual	Group definitions and tier mappings are manual configuration. The engine does not autodiscover groups or infer tier from Entra ID properties.

13. Project Roadmap

The validation engine is a companion system to the JML Identity Lifecycle Engine. It runs as a separate Azure Function application and is called by the JML pipeline at each provisioning event. The two systems share the same `sod_policies.json` conflict catalogue and the same `Rules.json` entitlement model.

The integration status table in Section 8 tracks the current state of JML pipeline coverage. Mover and Leaver support require additional gate logic in the validation engine to handle delta states and post-revocation verification. Both are designed and scheduled after the JML Mover phase build completes.

The longer-term path for SoD enforcement is a Python timer function that calls `sod_checker.py` directly, eliminating the four-line PowerShell duplication of the ANY_TO_ANY intersection check. Policy definitions remain in `sod_policies.json` regardless of which runtime evaluates them.

Henry Ezihe | Identity and Access Management Engineer | May 2026